

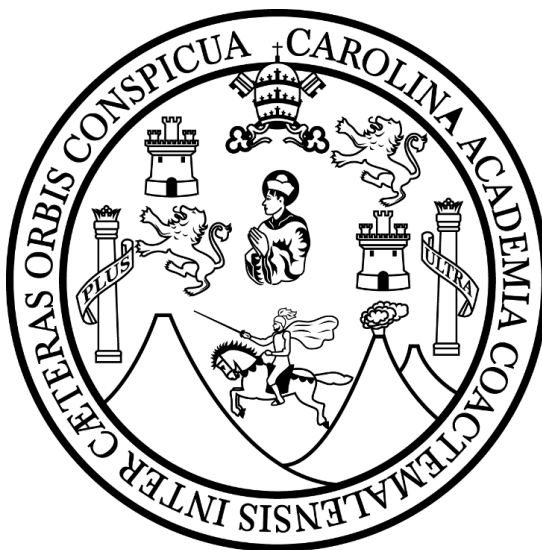
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

ESCUELA DE CIENCIAS Y SISTEMAS

INTELIGENCIA ARTIFICIAL 1

SECCIÓN A



Manual Tecnico

Proyecto Fase 3 Grupo 9	
201504464	Herberth Abisai Avila Ruiz
202000562	Pedro Luis Pu Tavico
201602983	Jaime Alejandro Armira Us

Tabla de Contenido

Introducción.....	3
Tecnologías y Herramientas Utilizadas.....	3
Estructura del Proyecto.....	3
Implementación del Modelo.....	3
Cómo Ejecutar el Proyecto.....	3
Documentación de Código.....	4
Mantenimiento y Mejora.....	4

Introducción

Este chatbot ha sido desarrollado para interactuar en tiempo real en español e inglés. Está diseñado para responder a preguntas relacionadas con la programación en JavaScript y Python, proporcionando ejemplos de código, además de ofrecer respuestas genéricas y conversaciones formales.

Tecnologías y Herramientas Utilizadas

- **Lenguaje de Programación:** Python (para la interfaz, el entrenamiento y manejo del modelo).
- **Bibliotecas:**
 - **TensorFlow.js:** Para la inferencia del modelo en el navegador.
 - **Keras:** Para el entrenamiento de la red neuronal.
 - **NLKT:** Para procesamiento del lenguaje natural
 - **CustomTkinter** para la interfaz gráfica de usuario.
 - **JSON:** Para manejar intenciones y patrones del chatbot.

Estructura del Proyecto

La estructura del proyecto está organizada en varias carpetas y archivos que manejan diferentes aspectos del chatbot:

- **Fase3/appv2.py:** Controla la interfaz gráfica del chatbot, donde el usuario puede interactuar con él.
- **Fase3/modelo/chatbot.js:** Código Python para preprocesar entradas y entrenar el modelo de red neuronal. Maneja las predicciones del modelo y la interacción con el usuario.
- **Fase3/modelo/training.py:** Código Python para el entrenamiento del modelo, generación de "bag of words" y clases.
- **Fase3/modelo/intents.json:** Archivos JSON (intents_es.json, intents_en.json) que almacena los intents (intenciones) y patrones de entrada, junto con las respuestas posibles.

Cada intent tiene un "tag" que representa la categoría de la consulta del usuario, un conjunto de "patterns" que son frases típicas que corresponden a ese intent, y las "responses" posibles que el chatbot puede dar. Todo esto para español e inglés.

- **Fase3/modelo/words.json, classes.json, chatbot_model.h5:** Archivos que contienen el modelo entrenado y las clases correspondientes.

Implementación del Modelo

A continuación se describen las partes clave:

1. **Preprocesamiento:** El texto de entrada es tokenizado, lematizado y convertido a una representación numérica usando una bolsa de palabras (bag of words).
2. **Entrenamiento:** El modelo se entrena con datos de intenciones (intents) que incluyen consultas sobre temas de programación en JavaScript y Python.
 - Las intenciones relacionadas con JavaScript incluyen:
 - `javascript_bubble_sort`: Proporciona ejemplos de Bubble Sort en JavaScript.
 - `javascript_merge_sort`: Explica cómo funciona Merge Sort en JavaScript.
 - Las intenciones relacionadas con Python incluyen:
 - `python_quick_sort`: Ejemplos de Quick Sort en Python.
 - `python_if_elif_else`: Ejemplos de estructuras condicionales en Python.

Cómo Ejecutar el Proyecto

1. Ejecuta el archivo `appv2.py` para iniciar la aplicación.
2. La interfaz gráfica se abrirá en una ventana y el chatbot estará listo para recibir entradas del usuario.
3. El chatbot proporcionará ejemplos y respuestas de acuerdo con las intenciones programadas.
4. O bien puedes ejecutarlo desde el ejecutable que se encuentra en la siguiente URL: [CHATBOT](#) entrar a la carpeta dist y ejecutar `./Chatbot`

Documentación de Código

Archivo `appv2.py`:

- Implementa la interfaz gráfica con CustomTkinter, gestionando la entrada y salida de mensajes del usuario y del chatbot.
- `__init__(self, root)`

Esta es la función principal del constructor de la clase `ChatApp`. Se encarga de inicializar la interfaz gráfica de la aplicación.

- **Configuración de la ventana:** Configura la ventana principal (`root`) de la aplicación, estableciendo tamaño, modo de apariencia y color de fondo.

```
self.root = root

self.root.resizable(True, True)

self.root.geometry("620x820+500+100")
```

- **Creación de widgets:** Define y organiza los elementos visuales como marcos (`Frame`), etiquetas (`Label`), campos de entrada de texto (`CTkEntry`), y botones (`CTkButton`).

```
self.msgInput = ct.CTkEntry(self.chatFram,
                             placeholder_text="Escribe el mensaje aqui...",
                             height=40, font=("arial", 18))

    self.msgInput.pack(side=LEFT, pady=10,
                       padx=10, fill=X, expand=True)

    self.submitBtn = ct.CTkButton(self.chatFram,
                                   text="", image=submitIconImage,
                                   bg_color=bgColorChatFram, fg_color=bgColorChatFram,
                                   height=40, width=20, hover_color=bgColorChatFram)

    self.submitBtn.pack(side=LEFT, padx=0,
                        pady=8)

self.submitBtn.configure(command=self.sendMessage)
```

- **Carga de imágenes:** Carga y redimensiona las imágenes de los iconos y avatares tanto para el usuario como para el chatbot.
- **Gestión de scroll:** Implementa un **Canvas** para mostrar los mensajes intercambiados entre el usuario y el chatbot, permitiendo el desplazamiento vertical.

```
self.canvas = tk.Canvas(root, bg='#f0f0f0',
highlightthickness=1)

self.label_frame = tk.Frame(self.canvas,
bg="#f0f0f0")

self.label_frame.pack(side="left", fill="both",
expand=True)

self.scrollable_window =
self.canvas.create_window((0, 0),
window=self.label_frame, anchor="nw")

self.canvas.pack(side=tk.TOP, fill=tk.BOTH,
expand=True)

self.label_frame.bind("<Configure>",
self.configure_scroll_region)

self.scrollbar = ct.CTkScrollbar(self.canvas,
orientation="vertical", command=self.canvas.yview)

self.canvas.configure(yscrollcommand=self.scrollbar.
set)

self.canvas.yview_moveto(1.0)
```

- **Configuración del botón de envío:** Asocia el botón de enviar mensaje con la función **sendMessage**, que manejará el envío de mensajes.

- **sendMessage(self)**

Esta función se ejecuta cuando el usuario envía un mensaje, ya sea presionando "Enter" o haciendo clic en el botón de enviar.

- **Obtención del mensaje del usuario:** Toma el texto ingresado en el campo de entrada y lo limpia después de enviarlo.
- **Visualización del mensaje del usuario:** Muestra el mensaje del usuario en la interfaz, añadiendo su avatar y el mensaje en un nuevo marco.
- **Consulta del chatbot:** Llama a la función `consulta()` (importada desde `modelo.chatbot`), que se encarga de procesar el mensaje y generar una respuesta.

```
try:

    self.to_respond = consulta(self.message)

    if self.to_respond:
```

- **Visualización de la respuesta del chatbot:** Si hay una respuesta, se muestra en la interfaz junto con el avatar del chatbot. La respuesta del chatbot también se puede copiar al portapapeles haciendo clic sobre ella, lo cual está manejado por la función `copy_bot_message`.
- **Manejo de errores:** Si no se ingresa un mensaje, se muestra un mensaje de error.

- `copy_bot_message(self, event)`

Esta función permite copiar el mensaje de respuesta del chatbot al portapapeles cuando el usuario hace clic sobre el mensaje.

- **Uso de `pyperclip`:** Copia el texto de la última respuesta del chatbot al portapapeles utilizando el módulo `pyperclip`.
- **Manejo de excepciones:** Si ocurre un error durante la copia, se muestra un mensaje de error.

```
def copy_bot_message(self, event):

    try:

        #message = event.widget._text # Usar
        _text en lugar de cget("text")
```

```

        pyperclip.copy(self.last_bot_response)

        messagebox.showinfo("Copiado", "Mensaje
copiado al portapapeles")

    except Exception as e:

        messagebox.showerror("Error", f"No se
pudo copiar: {str(e)}")

```

- **configure_scroll_region(self, e)**

Ajusta la región desplazable del **Canvas** donde se muestran los mensajes, de modo que el usuario pueda desplazarse y ver todos los mensajes.

- **Actualización de la región de scroll:** Recalcula el área desplazable de la ventana en función de los mensajes añadidos.

```

def configure_scroll_region(self, e):

self.canvas.configure(scrollregion=self.canvas.bbox(
'all'))

```

- **resize_frame(self, e)**

Ajusta el tamaño del marco que contiene los mensajes del chatbot cuando la ventana cambia de tamaño.

- **Redimensiona el contenido:** Ajusta el ancho del **scrollable_window** para que se adapte al nuevo tamaño de la ventana principal.

```

def resize_frame(self, e):

self.canvas.itemconfigure(self.scrollable_window,
width=e.width-40)

```


Archivo `chatbot.py`:

Este archivo contiene funciones como `bag_of_words`, que convierte las entradas de texto en una representación binaria, y `predict_class`, que predice la intención del usuario basada en el modelo entrenado.

- **`bag_of_words(sentence)`**: Convierte las palabras de una oración en una representación de "bag of words".

```
#Convertimos la información a unos y ceros según si están presentes en los patrones

def bag_of_words(sentence):

    sentence_words = clean_up_sentence(sentence)

    bag = [0]*len(words)

    for w in sentence_words:

        for i, word in enumerate(words):

            if word == w:

                bag[i]=1

    print(bag)

    return np.array(bag)
```

- **`predict_class(sentence)`**: Predice la intención de una oración basada en el modelo entrenado.

```
#Predecimos la categoría a la que pertenece la oración

def predict_class(sentence):

    bow = bag_of_words(sentence)

    res = model.predict(np.array([bow]))[0]

    max_index = np.where(res == np.max(res))[0][0]

    category = classes[max_index]
```

```
return category
```

- **get_response(tag, intents_json):** Selecciona una respuesta basada en la intención predicha.

```
#Obtenemos una respuesta aleatoria

def get_response(tag, intents_json):

    list_of_intents = intents_json['intents']

    result = ""

    for i in list_of_intents:

        if i["tag"]==tag:

            result = random.choice(i['responses'])

            break

    return result
```

Archivo **training.py**:

Define y entrena la red neuronal utilizada para clasificar las intenciones del usuario.

- **expand_contractions(text, contractions_dict):** Expande las contracciones en una oración.

```
def expand_contractions(text, contractions_dict):

    pattern = re.compile(r'\b(' + '|'.join(re.escape(key)
for key in contractions_dict.keys()) + r')\b')

    return pattern.sub(lambda match:
contractions_dict[match.group()], text)
```

- **Red Neuronal:** Implementa una red neuronal densa para la clasificación de intenciones.

```
# agregando mas neuronas al proyecto

model = Sequential()
```

```

model.add(Dense(1024, input_shape=(len(train_x[0]),),
name="input_layer", activation='relu'))

model.add(BatchNormalization())

model.add(Dropout(0.5, name="dropout_1"))

model.add(Dense(512, name="hidden_layer1",
activation='relu'))

model.add(BatchNormalization())

model.add(Dropout(0.5, name="dropout_2"))

model.add(Dense(256, name="hidden_layer2",
activation='relu'))

model.add(BatchNormalization())

model.add(Dropout(0.5, name="dropout_3"))

model.add(Dense(128, name="hidden_layer3",
activation='relu'))

model.add(BatchNormalization())

model.add(Dropout(0.5, name="dropout_4"))

model.add(Dense(len(train_y[0]), name="output_layer",
activation='softmax'))

```

- **Entrenamiento:** Ajusta el modelo utilizando las frases y patrones de entrada.

```

#Entrenamos el modelo y lo guardamos

model.fit(np.array(train_x), np.array(train_y),
epochs=500, batch_size=8, validation_split=0.2,
callbacks=[early_stopping, reduce_lr])

model.save("chatbot_model.keras")

model.summary()

```

```
tfjs.converters.save_keras_model(model, 'model_js')
```

Mantenimiento y Mejora

- **Entrenamiento adicional:** Para mejorar el modelo, se puede añadir más datos al conjunto de entrenamiento y ajustar los parámetros de la red neuronal.
- **Mejora del rendimiento:** Optimizar el tamaño del modelo y ajustar los hiperparámetros para mejorar su desempeño en dispositivos con recursos limitados.

Entrenamiento del Modelo

El modelo de chatbot fue entrenado utilizando **Python** y la biblioteca **Keras** para crear una red neuronal que clasifica las intenciones del usuario en función de las entradas proporcionadas. El proceso de entrenamiento incluye la preprocesamiento de texto y la conversión de las oraciones en una representación numérica que la red puede entender.

Pasos para el Entrenamiento del Modelo:

1. **Carga de Intenciones:**
 - Se combinan las intenciones definidas en los archivos `intents_es.json` y `intents_en.json` mediante la función `load_intents` del archivo `training.py`. Estos archivos contienen los patrones de entrada (frases comunes) y las posibles respuestas que el chatbot puede proporcionar.

```
def load_intents(files, dir_path):  
  
    combine_intents={"intents":[]}  
  
    for file in files:  
  
        with open(dir_path+os.sep+file, 'rb') as f:  
  
            intents = json.load(f)
```

```

combine_intents["intents"].extend(intents["intents"])

    return combine_intents

dir_path='./file_intents'

files_list = os.listdir(dir_path+os.sep)

intents = load_intents(files_list, dir_path)

```

2. Preprocesamiento del Texto:

- Se utilizan las bibliotecas **NLTK** y **WordNetLemmatizer** para tokenizar las oraciones, lo que significa dividir cada oración en palabras individuales.
- El texto se lematiza, es decir, se reduce cada palabra a su forma raíz. Por ejemplo, las palabras "corriendo", "corre" y "correr" se convierten en "correr" para uniformizar el análisis.
- También se expanden las contracciones utilizando el archivo **contractions.json**, donde se definen contracciones comunes (por ejemplo, "I'm" se expande a "I am").

```

words = [lemmatizer.lemmatize(word) for word in words if
word not in ignore_letters]

words = sorted(set(words))

pickle.dump(words, open('words.pkl', 'wb'))

pickle.dump(classes, open('classes.pkl', 'wb'))

```

3. Representación de "Bag of Words":

- Cada oración es convertida a una representación de "bag of words" en la que se crea un vector de unos y ceros indicando la presencia o ausencia de palabras específicas en la oración. Esta representación simplifica las oraciones para que puedan ser procesadas por la red neuronal.

4. Definición de la Red Neuronal:

- El modelo se define utilizando la API **Keras**. La red tiene las siguientes características:
 - Una capa de entrada con 512 neuronas y activación **ReLU**.
 - Una capa oculta con 256 neuronas y activación **ReLU**.
 - Una capa de salida con tantas neuronas como clases de intenciones, usando la función de activación **Softmax** para obtener probabilidades.
 - Se utilizan capas de **Dropout** y **Batch Normalization** para evitar el sobreajuste y mejorar la estabilidad del entrenamiento.

5. Compilación y Optimización:

- El modelo utiliza el optimizador **SGD** (Gradiente Descendente Estocástico) con un decaimiento exponencial de la tasa de aprendizaje para ajustar el modelo de manera eficiente. La función de pérdida utilizada es **Categorical Crossentropy**, ya que se trata de un problema de clasificación.
- Se incluye una estrategia de **Early Stopping** para detener el entrenamiento cuando el modelo deja de mejorar, evitando así el sobreajuste.

```
#Creamos el optimizador y lo compilamos

sgd = SGD(learning_rate=lr_schedule , momentum=0.96,
nesterov=True)

model.compile(loss='categorical_crossentropy', optimizer =
sgd, metrics = ['accuracy', 'Precision', 'Recall'])

# Definimos el callback de early stopping para evitar el
sobreajuste

early_stopping = EarlyStopping(monitor='val_loss',
patience=250,
restore_best_weights=True,min_delta=0.00001,verbose=1,
mode='min')

#reduce lr on plateau

reduce_lr = ReduceLROnPlateau(monitor='val_loss',
factor=0.2, patience=50, min_lr=0.0001)
```

6. Entrenamiento y Validación:

- El modelo se entrena utilizando las frases preprocesadas y sus respectivas etiquetas (intenciones). Se utilizan 500 épocas y un tamaño de lote de 8 ejemplos por iteración. Durante el entrenamiento, se utiliza un 20% de los datos como conjunto de validación para monitorear el rendimiento del modelo en datos no vistos.

```
#Entrenamos el modelo y lo guardamos

model.fit(np.array(train_x), np.array(train_y),
          epochs=500, batch_size=8, validation_split=0.2,
          callbacks=[early_stopping, reduce_lr])

model.save("chatbot_model.keras")

model.summary()

tfjs.converters.save_keras_model(model, 'model_js')
```

7. Archivos Generados:

- **words.pkl**: Contiene las palabras únicas usadas en el entrenamiento.
- **classes.pkl**: Contiene las clases (intenciones) identificadas.
- **chatbot_model.keras**: El modelo entrenado que se utiliza para la inferencia en tiempo real.
- **model.json**: La versión de TensorFlow.js del modelo para su uso en el navegador.

Crear ejecutable para el Chatbot

Requisitos previos

Instalación de PyInstaller: Asegúrate de tener instalado PyInstaller en tu entorno de Python. Si no lo tienes, puedes instalarlo ejecutando el siguiente comando en la terminal o consola:

```
pip install pyinstaller
```

1. **Especificación del archivo chatbot.spec:** El archivo `chatbot.spec` define cómo PyInstaller debe empaquetar el proyecto. En este archivo se incluyen las rutas a los archivos necesarios (como los modelos y recursos del chatbot), las librerías adicionales requeridas (como `tensorflow`, `PIL`, etc.) y los parámetros específicos para la creación del ejecutable.

Creación del ejecutable

Sigue los pasos a continuación para generar el archivo ejecutable:

1. **Preparar el entorno:** Asegúrate de que todos los archivos necesarios (como los modelos y datos del chatbot) estén ubicados en sus carpetas correspondientes y que el código `appv2.py` esté actualizado.

Ejecutar PyInstaller: Ejecuta el siguiente comando en tu terminal o consola dentro del directorio donde se encuentra el archivo `chatbot.spec`:

```
pyinstaller --clean chatbot.spec
```

2. Este comando generará un directorio `dist/Chatbot/` que contendrá el archivo ejecutable `Chatbot.exe`, junto con todas las dependencias necesarias.
3. **Opciones de PyInstaller:**
 - `--clean`: Elimina los archivos temporales generados por compilaciones anteriores.
 - `chatbot.spec`: Especifica el archivo de configuración que PyInstaller usará para empaquetar el proyecto.

Distribución del ejecutable

Una vez completado el proceso de empaquetado, el ejecutable estará disponible en la carpeta `dist/Chatbot/`. Puedes distribuir esta carpeta completa, que incluye el ejecutable y todas las dependencias necesarias para que el chatbot funcione en otros equipos sin necesidad de instalar Python ni otras bibliotecas.

Solución de problemas comunes

- **Error de importación de bibliotecas:** Si encuentras errores relacionados con librerías que no se encuentran, asegúrate de que las librerías necesarias estén especificadas en el apartado `hiddenimports` de `chatbot.spec`.
- **Faltan archivos de datos:** Si el ejecutable no puede encontrar algunos archivos de modelo o recursos, revisa que estén correctamente ubicados en el directorio especificado en `datas` dentro de `chatbot.spec`.